

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	Total Marks:	100
CO2 Statement:	<i>Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)</i>		
Student Name:		Reg. No.:	
Section / Batch:		Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions
Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q1

SECTION A – DEBUGGING & OPTIMISATION TASK

Code of Conduct

I, _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, wellstructured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%				
Debugging	25%				
Optimization	20%				
Analysis	20%				
Code Quality	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____

Signature: _____

Signature: _____

Signature: _____

Date: _____

Date: _____

Date: _____

CO'2'-AT'3'

Debugging and Optimization Task

Debugging Maximum–Minimum Finder Code (Else Branch Prevents Dual Update Logic)

1. Introduction

Debugging is the process of identifying, analyzing, and correcting errors or weaknesses in a program to ensure that it works correctly under all conditions. It is one of the most important stages of software development because even a small logical mistake can lead to incorrect results or unexpected program failures.

Optimization is the process of improving the efficiency, readability, maintainability, and reliability of a program without changing its expected output. Optimized code is easier to understand, simpler to modify, and performs better when processing large amounts of data.

In this scenario, a Python function is used to determine the **minimum** and **maximum** values in a list. Although the original implementation works correctly for most inputs, its use of an if...elif structure reduces code clarity and makes future debugging more difficult. The function also fails when an empty list is provided because no input validation is performed.

The objective of this assessment is to analyze the existing implementation, identify its weaknesses, debug the program systematically, optimize the logic, and produce a more reliable solution while maintaining efficient performance.

3. Existing Code

```
def min_max(arr):  
    mn = arr[0]  
    mx = arr[0]  
  
    for x in arr[1:]:  
        if x < mn:  
            mn = x  
        elif x > mx:  
            mx = x  
  
    return mn, mx
```

4. Understanding the Existing Logic

The function begins by assuming that the first element of the list is both the minimum and maximum value.

It then scans the remaining elements one by one.

For every element:

- If it is smaller than the current minimum, the minimum is updated.
- Otherwise, if it is larger than the current maximum, the maximum is updated.

Finally, the function returns both values.

Although this implementation usually produces correct results, the structure is not ideal for debugging or future modifications.

5. Problems Identified During Debugging

5.1 No Validation for Empty Lists

The function immediately accesses the first element of the list.

```
mn = arr[0]
```

If the input list is empty,

```
arr = []
```

Python generates:

IndexError: list index out of range

A robust application should validate the input before accessing any element.

5.2 Reduced Readability

The function uses an if...elif structure.

```
if x < mn:
    mn = x
elif x > mx:
    mx = x
```

When the first condition is true, Python automatically skips the second condition.

Although a value cannot normally become both a new minimum and a new maximum during the same iteration, using separate conditions makes the program easier to understand and reason about.

5.3 Difficult to Maintain

Software is rarely written once and never changed. Future developers may need to:

- Count how many times the minimum changes.
- Count how many times the maximum changes.
- Log every comparison.
- Add debugging messages.
- Record statistical information.

Independent conditions make these enhancements much easier.

5.4 Poor Error Handling

The original program assumes valid input.

It does not check for:

- Empty lists
- Invalid data
- Incorrect input types

A well-designed program should detect such situations and provide meaningful error messages.

6. Systematic Debugging Process

The debugging process was performed in several steps.

Step 1 – Study the Existing Program

Understand how the variables mn and mx are initialized and updated.

Step 2 – Identify Weaknesses

Test the program using:

- Empty list
- Single-element list
- Duplicate values
- Negative numbers
- Mixed positive and negative numbers

Step 3 – Modify the Logic

Replace the if...elif structure with two independent if statements.

Step 4 – Improve Input Validation

Check whether the list is empty before processing.

Step 5 – Test the Corrected Program

Run multiple test cases to verify correctness.

7. Optimized Code

```
def min_max(arr):
    """
    Returns the minimum and maximum values in a list.

    Parameters:
        arr (list): List of numeric values

    Returns:
        tuple: (minimum, maximum)

    Raises:
        ValueError: If the input list is empty.
    """

    # Validate input
    if not arr:
        raise ValueError("Input list cannot be empty.")

    # Initialize minimum and maximum
    mn = arr[0]
    mx = arr[0]

    # Traverse remaining elements
    for x in arr[1:]:

        # Update minimum independently
        if x < mn:
            mn = x
```

```

        # Update maximum independently
        if x > mx:
            mx = x

    return mn, mx

# Example
numbers = [12, 45, 7, 89, 23, -5, 67]

minimum, maximum = min_max(numbers)

print("Input List :", numbers)
print("Minimum Value :", minimum)
print("Maximum Value :", maximum)

```

Output

```

Input List : [12, 45, 7, 89, 23, -5, 67]
Minimum Value : -5
Maximum Value : 89

```

8. Why is the Optimized Code Better?

Independent Conditions

Using two if statements makes each comparison independent.

This improves readability and simplifies debugging.

Input Validation

The program now checks for an empty list before accessing the first element.

Instead of crashing, it raises a meaningful exception.

Better Documentation

The function contains a docstring explaining:

- Purpose
- Parameters
- Return values
- Exceptions

This improves code maintainability.

Easy Future Expansion

The optimized version can easily be extended with:

- Logging
- Counters
- Performance monitoring
- Statistical calculations
- 9. Testing the Program

Test Case	Input	Expected Output
Empty List	[]	ValueError
Single Value	[10]	(10,10)
Positive Numbers	[5,2,8,1]	(1,8)
Negative Numbers	[-5,-8,-1]	(-8,-1)
Duplicate Values	[7,7,7]	(7,7)
Mixed Values	[-2,4,0,9]	(-2,9)

10. Time Complexity Analysis

Let n be the number of elements.

The algorithm visits each element only once.

Each iteration performs at most two comparisons.

Therefore,

Time Complexity = $O(n)$

The running time grows linearly with the input size.

11. Space Complexity Analysis

The algorithm stores only two additional variables:

- mn
- mx

No extra arrays or additional memory proportional to the input size are required.

Therefore,

Space Complexity = $O(1)$

12. Advantages of the Optimized Solution

- Simple and easy to understand.
- Better code readability.
- Easier debugging.
- Better maintainability.
- Handles empty lists safely.

13. Conclusion

The debugging and optimization process significantly improves the quality of the maximum–minimum finder function. The optimized implementation validates input before processing, preventing runtime errors caused by empty lists. Replacing the if...elif structure with two independent if statements makes the update logic clearer, easier to debug, and more maintainable. Proper documentation and structured coding practices further enhance readability. The final solution maintains an efficient **$O(n)$** time complexity and **$O(1)$** space complexity, making it suitable for practical applications that process large datasets while remaining reliable and easy to maintain.